

Beyond RTL Simulation

Chapters 17–20. Acceleration and emulation; FPGA prototyping and hardware/software co-verification; gate-level, timing and power-aware simulation; post-silicon validation.

Four platforms. One question at each: what does this platform let me see, and what does it charge me for seeing it?

What you will be able to do

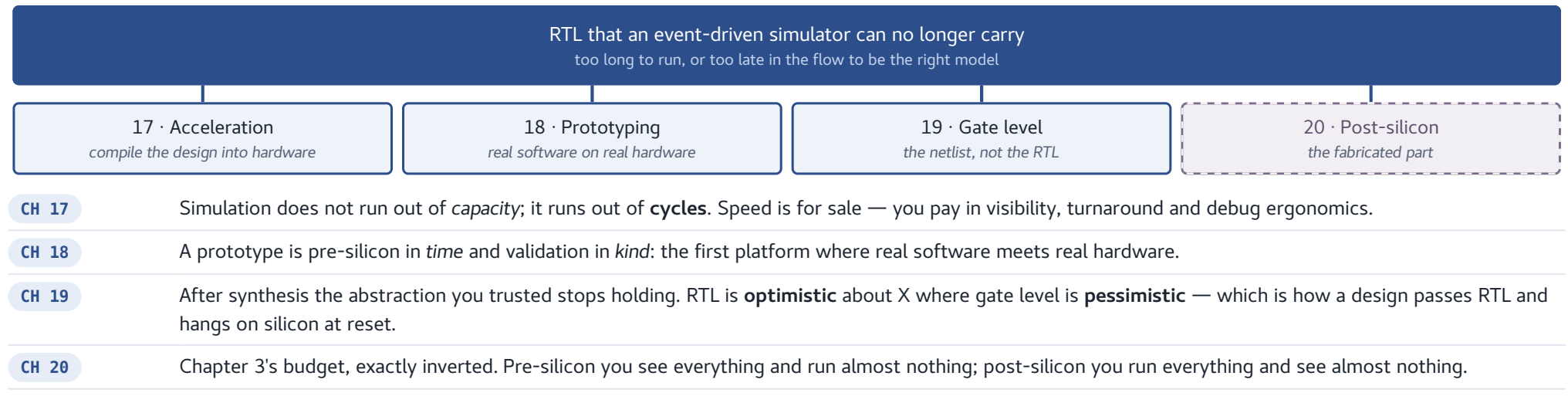
- Ask the one triage question that decides whether you need an emulator: *am I short of runs, or short of cycles in one run?*
- Place a transactor boundary where it pays, and predict what fraction of a machine's peak rate you will get.
- Say what an FPGA prototype establishes that no model can, and what it structurally cannot see.
- Scope a gate-level campaign that exhibits failures — and refuse the one that claims to sign off their absence.
- Design observability months before the failure exists, and localise a bug you can run but cannot see.

THE REGISTER SHIFT

Parts I–IV asked *have we checked enough?* Part V asks a harder pair: *can this platform even run the thing I need to check, and if it runs it, will I be able to tell what happened?* Every chapter here trades one for the other.

SAY *Nobody buys an emulator because simulation is slow. They buy it because one workload has no simulation answer at any farm size.*

Four chapters, four inversions of something you already believe



SAY Read the fourth row twice. Every habit Parts I to IV built assumes the first half of it.

17

Acceleration and Emulation

Compile the design into hardware and it runs a hundred to a thousand times faster. Everything else about the way you work has to change to pay for it.

Boot the operating system before tape-out. Now price it.

Simulation with RTL models is approximately **a billion times slower** than the target clock of the system it models. Map the same RTL onto a fabric or a purpose-built emulator and it runs about **100–1000×** faster than the simulator.

THE SAME ACTIVITY, TWO PLATFORMS

WHAT THE SOURCE REPORTS

An operating-system boot — seconds on target silicon — on an **RTL simulator**

several years

The same boot on an **accelerated platform**

back inside a few hours

DO NOT DIVIDE THESE

They are **two independent anchors** quoted from one source, not a ratio and its quotient. Divide "several years" by the 100–1000× band and you manufacture a third figure that composes with neither. Quote both; the argument turns on *years vs hours*, and no reordering of the numbers moves that boot into a nightly window.

SAY Read all of these as order-of-magnitude claims — the band spans a factor of ten and where you land depends on design size, how much is instantiated, and how much instrumentation you compiled in.

The gap is structural, and buying machines does not close it

Why the ratio exists

An event-driven simulator turns the design's parallelism into its own serialism: every signal change is a software event, and every process sensitive to it is re-evaluated in sequence. A million gates switching in one design cycle are a million pieces of work for one processor.

Why it stopped improving

For two decades the industry rode processor frequency. Frequency scaling plateaued, and simulation stopped keeping pace with design complexity. The obvious escape does not exist: an individual RTL simulation does not usefully exploit multiple cores.

The parallelism fallacy

"We will add machines." A farm's width multiplies *runs*; it does not shorten a *run*. Cycle n of a boot depends on cycle $n-1$, so one run has no parallel fraction to exploit.

THE TRIAGE QUESTION

Is my problem the number of runs, or the length of one run? Ask it out loud before any conversation about capital equipment. Only the second answer leads to Chapter 17.

And the arithmetic is per *single* execution. The flagship's nightly candidate list is 6,400 simulations totalling 4,409 CPU-hours — already re-tiered and pruned to fit a night. Accurate metastability injection degrades simulation a further 3–4×.

SAY A 4,800-run random suite is a licence problem. One boot is a cycles problem. Same word "slow", two unrelated diseases.

Two generations, one verification problem — and the interesting result is what *does not* move

PROBLEM	REFERENCE SOC	FLAGSHIP SOC
DMA 4 KB legalization, 2D transfers, negative stride	Constrained-random simulation, multi-seed	<i>Unchanged</i> — constrained-random simulation, more seeds, eight channels
Register and peripheral suites	Simulation	<i>Unchanged</i> — carried forward
Boot to a working system	Bare-metal image, a few million cycles, one nightly run	Operating system; no simulation answer at any farm size
Interconnect congestion behaviour	Crossbar arbitration, proved and simulated at block level	Mesh NoC latency and occupancy statistics under sustained load

SCENARIO · THE SHAPE THAT HAS NO SIMULATION ANSWER

A video codec’s rate-control loop adapts quantisation to a rolling estimate of channel occupancy. A few macroblocks per run match the reference model exactly; in the lab, streams drift out of bitrate after a scene change followed by a long static passage. The bug is not a data corner more seeds would find — it is a control loop with hundreds of frames of memory. **Randomising harder explores the wrong axis; only length reaches it.**

SAY *Emulation is not "simulation but faster for everything". Buy it for the other shape and you own an idle machine and an unfinished regression.*

The comparison that matters is not speed — it is the three things you hand over to get it

	SIMULATOR	EMULATOR	PROTOTYPING PLATFORM
Throughput	baseline	10^2 – $10^3 \times$ the simulator	the fastest pre-silicon platform
Capacity	host memory and patience	order of a billion gates	device capacity; above it, multi-FPGA partitioning
Build time	minutes	hours to days for a complex design	hours for a bitstream
Visibility	any internal signal, at any time	triggers, selective trace, save/restore	a few thousand signals, chosen <i>before</i> the build
Changing what you observe	edit and re-run	re-arm a trigger; recompile if never brought out	recompile the bitstream
Parallel width	licences (hundreds)	capacity $\div$ design size	capacity $\div$ design size, per board

TURNAROUND IS THE COST THAT RESHAPES THE WORKFLOW

In simulation you edit, rebuild in minutes, and can afford to be wrong about what to look at ten times a day — the reference SoC's smoke tier is under nine minutes end to end. On an emulator you may get **one build per day**. Every question you did not think to ask before the build costs a day — so engineers front-load: compile in the assertions and monitors, and treat "which signals come out" as a review decision, not a debug reflex.

SAY *The build is paid once and divided by everything that runs on it. One run per build is indefensible; two hundred makes it a rounding error.*

In-circuit or virtual: the environment decides, not the feature list

In-circuit (ICE) — the environment is the world

The design runs at MHz; a real link runs at its specified rate and has timeouts. A **speed adapter** buffers, throttles and re-times between them — the quiet asterisk on every ICE fidelity claim: *real content at manufactured timing*.

buys stimulus nobody could write (a real driver stack, quirks and all); interoperability against third parties you have no model of; compliance procedures that exist only as instruments.

forecloses repeatability (no seed, nothing to replay — and Chapter 12's whole triage apparatus assumes a run is a function of recorded inputs); scheduling; unattended operation; save/restore, because you cannot rewind the world.

Virtual — the platform owns model time

Built on SCE-MI, the platform generates clocks *on the emulator* and tracks model simulation time inside it. Own model time and you can stop it, resume it, replay it, and place an event at an exactly chosen instant.

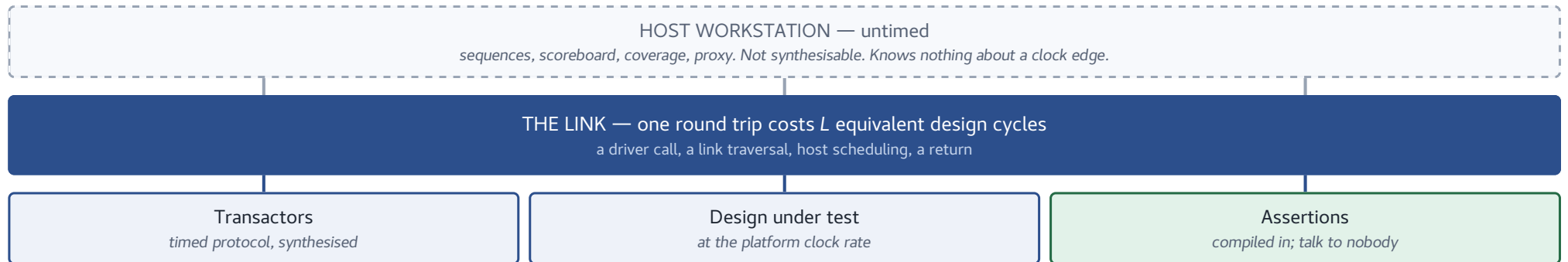
Everything the industry wanted follows: deterministic reruns, controlled clock relationships, injection at a specified cycle, waveform display, and an elimination strategy — disable a subset of injection points, rerun the identical scenario, see if the failure persists.

THE RULE

Use **ICE when the environment is the thing you cannot model**; virtual for everything else. A compliance instrument, a third-party host, a driver stack you do not own — those are *environments*. Noise, traffic patterns and workloads are *data*: recordable once, replayable forever, and paying the irreproducibility tax for nothing.

SAY The USB compliance suite closes the protocol-state half of its vplan rows. Timing at the pins belongs to the adapter, not the design — those rows stay open until silicon.

The transactor stops being a convenience and becomes a physical cut



THE FAILURE MODE HAS A NAME

Accelerate the DUT and change nothing else: the protocol is still in the UVM driver, so **every pin event crosses the link**. That is *signal-level acceleration only*. You have bought a machine that runs a billion gates at MHz and are feeding it one clock edge at a time from a workstation.

Read the figure for what is **absent above the boundary**: clock and reset generation, and the assertions. Nothing on the host knows what a clock edge is; nothing on the host is consulted per cycle. The scoreboard is fed at *transaction* rate by a monitor transactor.

A UVM component is not synthesisable, so it cannot cross. The question is not *whether* to have a boundary — it is where to draw it. Answer: at the transaction.

SAY Chapter 8 warned about this in advance: an environment that reaches into signals from twenty places cannot be moved at all.

Where you draw it, not the headline frequency, sets your throughput

f = platform clock rate. N = design cycles advanced between host interactions. L = cost of one host interaction, in equivalent design cycles. Each period delivers N cycles and costs $N + L$:

$$R(N) = f \cdot N / (N + L)$$
$$S(N) = R(N)/R(1) = N (1 + L) / (N + L)$$

S saturates at $1 + L$ however large N grows — batching can never buy more than the round-trip cost itself — and reaches exactly **half** that ceiling at $N = L$.

THE DESIGN RULE

Not "use transactions". **Make transaction granularity comparable to your round-trip cost** — and stop optimising once it is.

Fraction of the machine's peak rate you actually get, $N/(N + L)$. L is the parameter you least control — take it as tens to thousands of cycles:

CYCLES PER HOST INTERACTION	$L=20$	$L=200$	$L=2000$
1 (pin-level)	4.8%	0.5%	0.05%
16 (a short burst)	44%	7.4%	0.8%
256 (a max-length burst)	93%	56%	11%

Pin-level throws away between **95% and 99.95%** of what you paid for. And the rightmost column is the warning against complacency: on a costly link even a full 256-beat burst leaves you at a tenth of peak. If your numbers look like that column, the answer is a *coarser* transaction — a descriptor, a page, a frame — not a faster link.

SAY This table is fraction of peak rate. It is not $S(N)$; the speed-up and the efficiency are different quantities with different ceilings.

The mechanical change is small, local, and entirely about who owns the clock

```
// Below the boundary: synthesised onto the platform.
module wr_xactor #(parameter int ADDR_W = 40, parameter int DATA_W = 512)
  (input logic clk, rst_n, awready, wready,
   output logic [ADDR_W-1:0] awaddr, output logic [7:0] awlen,
   output logic [DATA_W-1:0] wdata,
   output logic awvalid, wvalid, wlast);
  logic [31:0] lfsr;
  export "DPI-C" task do_write_burst;
  task do_write_burst(input longint unsigned addr,
                     input byte unsigned len, input int unsigned seed);
    lfsr = seed; @(posedge clk); while (!rst_n) @(posedge clk);
    awaddr <= ADDR_W'(addr); awlen <= len; awvalid <= 1'b1;
    do @(posedge clk); while (!awready);
    awvalid <= 1'b0;
    for (int i = 0; i <= int'(len); i++) begin
      lfsr = {lfsr[30:0], lfsr[31] ^ lfsr[21] ^ lfsr[1] ^ lfsr[0]};
      wdata <= {(DATA_W/32){lfsr}};
      wlast <= (i == int'(len)); wvalid <= 1'b1;
      do @(posedge clk); while (!wready);
    end
    wvalid <= 1'b0; wlast <= 1'b0;
  endtask
endmodule
```

SAY Above the boundary an interface imports the same task and the driver becomes one call. **Anything the environment needs every cycle becomes hardware** — that sentence predicts how much of a testbench has to be rewritten.

Four load-bearing things

- **The task is timed; the call is not.** Everything waiting on `clk` is inside the module. *Do not use clocks to synchronise with the testbench* — a driver that waits for an edge has re-created the pin-level case by another route.
- **One call covers `len + 1` beats.** At `awlen`'s maximum of 255, that is the $N = 256$ row of the table for one host interaction.
- **One source serves both engines.** A simulator executes this directly, so a bug found on either platform reproduces on the other — as long as the two remain one source.
- **The payload is manufactured on the platform.** The host passes a 32-bit seed; the shift register makes the data. Shipping a 512-bit beat 256 times is 256 interactions again.

A licence pool degrades gracefully under contention. An emulator is granted, or it is not.

N runs cost $N \cdot s / W_{\text{sim}}$ on the farm and $B + N \cdot r / W_{\text{hw}}$ on hardware. The forgotten term is W_{hw} : **hardware parallel width is capacity divided by design size**, so a full-SoC image gives $W_{\text{hw}} = 1$.

FLAGSHIP WORKLOAD	THE ARITHMETIC
OS boot	N a handful per revision; s years; r a few hours; B hours to days. Break-even below one run : it does not need a business case, it needs a slot.
DMA legalization suite	4,800 runs, 30-minute mean, 2,400 CPU-hours, ten hours of wall clock on 240 licences . Even at $r < 2$ s, $W_{\text{hw}} = 1$ gives about 2.4 hours of machine time — then add B , paid <i>per design revision</i> . A nightly suite runs per revision, so it buys nothing.

- 1. **No simulation answer**: boot, firmware, long soaks, NoC statistics.
- 2. Campaigns whose value is **iteration count under one image**, where the build amortises across the sweep.
- 3. **Reproductions of failures found downstream**: the platform has the observability silicon lacks.
- 4. **Nothing a farm can do** — the rule always broken.

ONE EXTRA VPLAN COLUMN – NOT THE OBVIOUS ONE
An emulation row must name **which image it runs on**. The image is the scheduling unit, so rows batch by image, not priority. *Plan the images first, then assign rows* — or discover, late, that the top-priority ones need five builds you have no calendar for.

SAY Three claims earn a slot and one rule protects it. A University of Florida survey states the rationing outright: *emulator capacity is often reserved for tape-out-critical functional regressions, so security and fault campaigns are cut to high-risk blocks or deferred.*
T. Rahman et al., *Emulation-based SoC Security Verification*, Univ. of Florida, arXiv 2604.15073 – [P14, pp. 8, 17]. Qualitative; names no platform.

18

FPGA Prototyping and HW/SW Co-verification

Pre-silicon in *time*; validation in *kind*. The first platform on which real software runs against real hardware — and the first on which the debug ergonomics of the lab decide what a bug costs.

The bug was not hard. The nine days were.

Four FPGA boards in a rack. Nine months before tape-out the flagship boots an operating system, mounts a filesystem over Ethernet, reaches a shell — and works for about **forty seconds**. Then a compute-cluster benchmark returns a result whose last cache line is stale. Run it again: passes. Run it forty times: **fails twice**.

What nobody schedules. Verification cannot reproduce it — their DMA and coherence tests pass and the failing workload is hundreds of millions of cycles long. Software's driver has run six months against a functional model without a failure. Available evidence: one console, a few LEDs, and a trace buffer configured for a different question three bitstreams ago.

NINE DAYS LATER

The flagship's DMA mis-legalizing a 2D descriptor at a 4 KB boundary — the reference SoC's canonical bug, inherited, and hidden one layer deeper by coherence, because the split's second half writes a line the host cluster holds Modified and the corruption stays invisible until the writeback.

THE TERMINOLOGICAL LINE THIS CHAPTER TURNS ON

Validation means checking a design on *real hardware* rather than on a model — post-silicon on fabricated parts, *or in the lab on a prototype*. So an FPGA prototype is **pre-silicon in time and validation in kind**. Not a faster simulator: a different *kind* of evidence, with the lab's debug economics attached.

SAY *Nine days of ownership argument, for a bug both teams already knew the shape of. Everything in this chapter is aimed at that number.*

Three properties, all doing work — and one collapse that pays for them

What the prototype is for

- **Hardware speed** — megahertz: roughly 100–1000× an RTL simulator, turning an OS boot from an impossibility into a few hours of runtime.
- **Real I/O** — the console is a real UART, the network a real PHY. The driver being exercised is *the* driver, not a testbench's idea of one.
- **Cycle accuracy** — no abstraction sits between the software and the design it programs.

THE SHAPE OF WHAT THAT BUYS

A reported 2026 flow runs an **FPGA test suite of 1,738 tests** in three classes — RTL-against-platform integration, bare-metal benchmark and litmus, and OS-level driver and stack tests. Inside it, a full Linux boot of a dual-core design clocked at **20 MHz completes in 115 seconds**. Two minutes, on a design far smaller than the flagship, for a boot RTL simulation would never finish.

The defining limitation is not speed or capacity

It is that **you cannot see inside it**. Observability is restricted to a few thousand internal signals; you must decide *which* before the bitstream is generated; and changing your mind means recompiling — several hours. Compare the simulator you left behind, where every net is visible for free, after the fact, without rerunning anything.

So prototyping emphasises system-level execution and functional realism, and deep debug is performed selectively or **deferred** to a higher-visibility environment. *Deferred* is the word to notice: it means reproducing the failure in simulation or on an emulator — exactly what the opening scenario's teams could not do.

Turnaround compounds it. Synthesis and partitioning take many hours or days, and probes are *logic*, competing for the same fabric and routing as the design. The loop that costs seconds in simulation — add a probe, rerun, look — costs a working day here.

SAY *What you can see is decided before the run, and changing your mind is a compile. That one sentence explains every workflow difference in this chapter.*

Two distortions the platform introduces, and the one thing it cannot host at all

Partitioning inserts machinery the design does not have

Bridges between partitions, serialiser/deserialiser links across the board, synchronisation logic to hold the pieces together — at the cost of communication latency across FPGA boundaries. A crossbar or NoC path costs materially more cycles across a partition boundary than inside one device.

STATE THIS BEFORE ANYONE QUOTES A NUMBER

Performance measured on a partitioned prototype is a **lower bound with an unknown gap**, never a projection. The arbitration and back-pressure behaviour you observe is not the behaviour silicon will have.

Clocking distorts selectively — derive it

The system clock drops by some factor to close timing in fabric, and the peripheral domain drops with it, so the reference SoC's **4:1** ratio (200 MHz : 50 MHz) *survives* — divide both by the same factor and the relationship holds. The always-on domain does not cooperate: its **32 kHz** reference is a real crystal on the board, and a crystal does not scale. The **6,250:1** ratio becomes 6,250 divided by the system slowdown factor.

The canonical wake-cause bug lives exactly there: a legal `3'b010 → 3'b100` transition briefly observed as the reserved code `3'b110`. Whether the prototype shows it depends on how many fast-domain sampling edges fall inside the slow domain's transition window — the very quantity you just changed. **A prototype that never reproduces a crossing bug and one that reproduces it constantly can be the same design at two clock settings.**

WRITE IT IN THE PLAN AS AN EXPLICIT PROTOTYPE-SCOPE EXCLUSION

The flagship's LPDDR4 controller and PHY — whose training state machine is among the riskiest things on the chip — are exactly what *cannot* be prototyped: the PHY is analog and its timing is the point. On the board they are replaced by the platform's own memory controller. The prototype will run the OS beautifully and tell you **nothing whatever** about memory training.

SAY Structural CDC analysis found the crossing bug in seconds. The prototype is not the instrument for it — in either direction.

Not "cheap emulator" — one instrument on each side of the verification/validation line

THE RIGHT MENTAL MODEL

An emulator is a fast simulator with reduced visibility. A prototype is early silicon with more visibility than silicon.

Work you would do in a simulator, if only it were faster, belongs on an emulator. Work you would do in the lab, if only you had a chip, belongs on the prototype.

The admission test — all three must hold

1. **It needs more cycles than simulation can deliver.** If it fits in a simulation run, run it there, where debug is cheaper.
2. **It has an oracle that survives the visibility collapse.** An offline bit-exact diff, self-checking firmware, an in-fabric monitor latching into a readable register. A workload whose only oracle was "the engineer watched the waveform" has no oracle at all here.
3. **Its failure is plausibly reproducible in a higher-visibility environment** — because that is where it will be debugged.

Condition 2 is the one teams skip; condition 3 is the one that costs them.

SCENARIO • A WORKLOAD THAT PASSES ALL THREE

The video codec's encoder passes its block environment: bit-exact per macroblock, coverage closed on modes, quantiser steps, buffer-full. What nobody has run is a *sequence* — a thousand consecutive frames where rate control adapts across scene changes.

Approach: run it on the prototype from a real clip, write the bitstream to storage, diff the whole output offline. **Why it works:** the cycles are genuinely needed; the oracle needs no visibility at all; and when frame 738 differs, the frame number and the encoder state are enough to build a simulation testcase. *That last property is what makes it worth doing — not the speed.*

THE COUNTER-CASE

In the FPGA-target segment, where teams routinely minimise pre-lab verification to reach the lab quickly, only **13 percent** of projects in the **2024 Wilson Research Group FPGA study** reported zero non-trivial escapes into production. **Prototypes make bugs visible late; they do not make them findable.**

SAY Trading a controllable, observable, reproducible environment for an uncontrollable, unobservable, weakly reproducible one — and gaining only speed. The same 2024 study pair puts IC/ASIC first-silicon success at 14%.

WRG / Siemens EDA 2024 trend reports — FPGA [R1, p.5]; IC/ASIC [R2, p.5]. Double-blind, $\pm 4\%$ at 95%.

Fast, indispensable, and structurally blind to one entire class of bug

establishes that the initialisation sequence sets the registers it means to, in an order the hardware accepts; that firmware's view of the memory map agrees with the architecture's; that handlers are wired to the sources they think; that device tree, linker script and boot ROM hand-off agree. On the flagship: four cores of OS bring-up, a driver stack and a filesystem, none of it waiting for RTL.

absent no arbitration — nothing to arbitrate. No back-pressure — a model that returns immediately never stalls. No latency — two operations the design will overlap are sequential. No coherence — a flat memory has nothing to be coherent with.

ABSENT, NOT UNDETECTED — WHICH IS WORSE

The entire class of bugs that consists of *two things happening at once* is missing, and the platform reports success with complete confidence.

Two traps, both producing firmware that passes and then fails

- **Assumptions the model never charged for.** A poll loop terminating on the first read against a model that completes instantly — so nobody notices it has no timeout and no memory barrier.
- **Synchronisation the model made unnecessary.** A buffer handed to DMA without cache maintenance works perfectly with no caches. It surfaces on the prototype looking exactly like a hardware coherence bug — and is argued about as one.

SCENARIO · THE DIAGNOSTIC SIGNATURE

A GPU driver, eight months green against a functional model, tears a frame once every few thousand submissions on the prototype. Do not start by suspecting the driver: ask *which of its assumptions the model was structurally unable to test*. The model had no concurrency to get wrong, so it could neither confirm nor deny the ordering between the command processor's descriptor read and the shader clusters' writes. It never posed the question. **When a prototype failure looks like a race, ask first what the software's development platform could not model.**

SAY A third trap: the model becomes the specification by default — neither it nor the RTL is golden. And the approximately timed middle setting is not silent about timing: it has a timing, plausible, quotable, and not the design's. It answers architecture questions, whose answer is a ratio; not verification questions, whose answer is a verdict.

Where you put the boundary *is* the experiment

Virtual platform on the host
fast instruction-set models: what you are NOT verifying

TLM channel
untimed hand-off

Engine executing selected RTL
cycle accurate: what you ARE verifying

The placement rule — three things true at once

1. **Already an architectural boundary with transaction semantics.** A register interface, a bus port, a mailbox. A boundary drawn mid-microarchitecture — across a pipeline, a credit counter, a handshake — has no transaction to send.
2. **The virtual side is mature.** You are trusting it, unverified, for the duration of every run. A model written last week to fill a hole in the partition is a second design with no test plan.
3. **Everything whose timing you care about is on the RTL side.**

WHAT THE BOUNDARY DELETES

The channel is untimed, so **every bug living in the timing relationship between the two sides is unreachable by construction.** If the core and the DMA contend for one memory port and only the DMA is RTL, the contention is *gone* — not modelled coarsely, gone. That is not a defect in hybrid platforms; it is what they are. It is a defect only when nobody writes it down.

So write it down, on the plan row: "cross-boundary timing between the host cluster and the compute island is out of scope on this platform; covered by block-level full-RTL simulation and by the prototype's system regression." A written exclusion is a coverage decision; an unwritten one is a hole with a green dashboard over it.

SAY Same problem, two scales, opposite answers: on the reference SoC full-RTL simulation of the whole chip is affordable and a hybrid channel buys nothing; on the flagship it is how the driver team runs real code against the real compute cluster in month four rather than month eleven.

Not a sequence that resembles what software will do — the binary that will ship

Constrained random asks *is every legal transaction handled correctly?* Firmware asks *does this sequence, in this order, from this reset state, achieve the effect the software needs?* — and answers for one trajectory. Orthogonal projections, and mainstream: in 2024, **84 percent** of IC/ASIC projects incorporated one or more embedded processors.

- **Initialisation ordering.** Access constraints and preconditions are frequently *not documented*, and required nonetheless. The driver's sequence is what silicon sees for the rest of its life.
- **Register access races.** Three driver variants checked against each other: a missing side-effect annotation introduced a race that could make the device raise an interrupt incorrectly — and it had passed the simulation-based verification previously applied.
- **Error-recovery paths.** Timeouts, retries, channel resets. Nobody writes directed tests for these; running the driver runs them anyway.
- **Integration wiring.** The interrupt on the wrong input; the address the header and the decode logic disagree about. Block environments passed, the system regression passed, and it surfaced in the lab as a hang.

TWO LIMITS — THE SECOND IS EASY TO MISS

Firmware is one path. It uses the burst lengths the driver chooses and the alignments the allocator produces. It will never issue the legal transaction the driver has no reason to issue, and never *varies* anything — a thin deep thread through a wide space. **And firmware is not an oracle for the hardware.** A driver checks that it achieved its own goal, not that the design behaved correctly. A hardware bug the retry logic silently absorbs is a passing boot with a defect in it.

SAY The crypto key ladder: once per few hundred re-keys a session decrypts to garbage, because the driver writes the next key-slot register while the previous operation is still draining — a requirement on the software, discovered in the hardware, written down in neither. Firmware-only ships a design that works for the driver it grew up with; random-only ships one where every transaction is legal and the initialisation sequence deadlocks.

On the prototype, that statement does not exist

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);

// The same expression, in fabric. first_len/first_size dropped whole.
module dma_4kb_monitor #(parameter int unsigned ADDR_W = 40) (
  input  logic clk, rst_n, awvalid, awready, input logic [7:0] awlen,
  input  logic [ADDR_W-1:0] awaddr, input logic [2:0] awsize,
  input  logic [1:0] awburst, output logic viol_seen,
  output logic [15:0] viol_count, output logic [ADDR_W-1:0] first_addr);
  logic viol;
  always_comb viol = awvalid && awready && (awburst == 2'b01) &&
    ((16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) > 16'd4096);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      viol_seen <= 1'b0; viol_count <= '0; first_addr <= '0;
    end else if (viol) begin
      if (!viol_seen) begin viol_seen <= 1'b1; first_addr <= awaddr; end
      if (viol_count != 16'hFFFF) viol_count <= viol_count + 1'b1;
    end
end
endmodule
```

Concurrent assertions are a simulation and proof construct. There is no assertion engine in the fabric. The check must become **logic that reports its verdict through a register the driver can read**.

What it costs

- **The waveform.** One address, one length, one size — and no picture of what the interconnect was doing around them.
- **Every violation after the first** — hence the counter. It separates *systematically illegal descriptors* from *one corner case fired once*.
- **Fabric.** Real logic, competing for the same routing as the design — the currency that buys debug probes.

WHAT IT KEEPS

Evaluated at the instant of the violation, at full speed, on every burst of a workload no simulator will reach. Detection distance: **zero cycles** — though the distance to *your knowledge of it* is however long until the driver next polls.

SAY *Build the monitors before the bitstream. After it, they cost a compile.*

Run the triage against the specification, not against the two implementations

TRIAGE OUTCOME	WHO OWNS IT
spec says X, hw does X	Software is wrong. Fix the driver.
spec says X, hw does Y	Hardware is wrong. Fix the RTL, and add the test that should have caught it.
the spec does not say	Neither team owns it. A specification defect — and treating it as a firmware bug is the single most common triage error in hardware/software co-verification.

1. **A run you can repeat.** A prototype run has no seed; the nearest equivalent is a manifest pinning RTL revision, bitstream hash, partitioning, clocks, image hashes, board, input data. *Repetition substitutes for replay*: the 2026 flow's indivisible OS boot is run **eight times** to show the result is stable.
2. **One timeline.** Three records of time in three formats means guessing whether the driver's timeout preceded the DMA's error.
3. **A path back to visibility** — a prototype failure's value is realised when it becomes a simulation testcase.

THE INSTANCE THAT INVERTS THE REFLEX

An open-source UART's baud counters run continuously. Reprogramming to a lower rate while the counter sits between the two leaves it un-reset until it wraps — unresponsive for up to **2¹⁸ clock cycles** — and **the software cannot avoid this**, because nothing lets it infer the counter's value. *No correct driver exists.*

SAY Arithmetic edition: a 16-bit "signed coefficient" the RTL reads as Q2.14 and the firmware computes as Q1.15. A spec defect filed as a firmware bug is closed by a driver patch — and the document stays wrong.

19

Gate-Level, Timing and Power-Aware Verification

After synthesis the abstraction you trusted stops holding. RTL is optimistic about X where the netlist is pessimistic — and only here can a simulator model losing power.

Same platform, same week, same engineers — one run was ritual, the other saved the tape-out

RESULT 1 — A BOOT TEST DIES AT TIME ZERO IN A FOG OF UNKNOWNNS

Nine hours of debug establish: the netlist's **scan-enable pin** was never driven by a testbench written against RTL, where no such pin existed; a scoreboard comparison written with `!=` had been **quietly reporting success on all-unknown data** for two of those hours; and a white-box checker on the DMA's internal descriptor pointer has nothing left to bind to, because synthesis dissolved the register it watched.

Nothing has been learned about the design. A great deal has been learned about the testbench.

RESULT 2 — THE ONE THAT SAVES THE TAPE-OUT

Run with the chip's **power intent loaded**, a sleep/wake test shows the event unit coming out of a power-down cycle with its interrupt-mask configuration destroyed.

The canonical escape of Chapter 7 — **a bug plain RTL simulation could not represent at all**, because plain RTL simulation does not model losing power.

A chapter on verification after synthesis is really a chapter on telling those two apart — because gate-level simulation is expensive enough that **running it out of habit is a decision with a price tag**.

SAY Ask of every proposed gate-level run: which of these two is it going to be? You can usually tell in advance.

Four arguments are made for gate-level simulation. You are owed the ranking, not the list.

THE ARGUMENT	VERDICT	WHY
"It checks that synthesis did not change the function"	weakest	Logic equivalence checking is exhaustive over the transformation and needs no stimulus. If you run LEC after synthesis — and you should — this argument is spent.
"It checks timing"	category error	STA examines every timing arc structurally, across corners, voltage, temperature and variation. A simulation samples one test's paths, at one corner.
"It finds X and reset problems the RTL hid"	strongest	if , case and edge constructs resolve an ambiguous condition into a definite branch — and no longer exist in a netlist. A design that passed at RTL on a silently resolved unknown fails here, at reset.
"It exercises structures not in the RTL at all"	underweighted	Scan chains, test-mode multiplexers, self-test wrappers, debug fabric — inserted after sign-off, so the obligation has no earlier home.

THE THESIS, IN MINIATURE

GLS with timing can **exhibit** one instance: one path, one corner, one alignment of edges. It cannot **cover** the class, because whether that alignment occurs is an accident of the stimulus — sign-off needs the space, and a structural analysis gives that. **Exhibiting a failure and signing off its absence are different jobs; GLS is qualified only for the first.**

SAY Scan insertion on the second flop of a two-flop synchronizer erodes MTBF where the first does not — a structural property of the implemented netlist that no simulation could have found.

Most first gate-level failures are not about the design at all

FAMILY	THE TELL	WHAT TO DO
Genuinely unreset state	Present at time zero, survives reset de-assertion, traces back to a flip-flop with no reset pin	The class GLS exists for — but findable earlier, and completely, by formal reset analysis.
Pins that exist only in the netlist	The cone of influence starts at a top-level input with <i>no RTL counterpart</i>	A missing harness nobody could have written earlier. Build one tie-off module; <i>review it as an artifact</i> .
Environment that cannot survive a netlist	The failure appears in the environment's own messages: elaboration error, null handle, back-door read returning unknowns	Synthesis dissolved, renamed or replicated the signal. What must survive is written against <i>ports</i> .
Pessimism artifacts	The unknown appears mid-logic with no source — an expression with partially unknown operands	Most debug time, least teaching. Clear the real sources first: each removal collapses a fan-out.

WHERE JUDGMENT ENTERS

The radar FFT's butterfly registers are deliberately unreset — pure datapath, overwritten by the first frame. The first run floods the chain with unknowns and a junior engineer opens a bug. It is not a bug, but it is not *obviously* not one, and the difference must be **argued**: no control decision consumes those registers before the flush completes. Pin the argument with an assertion.

THE ORDER OF OPERATIONS

Timing annotation, power intent and netlist stage are **three independent switches**; teams turn them on together, then debug the interaction. Come up against a *zero-delay netlist*, *no power intent*, on the shortest reset-and-boot test, and add nothing until it is clean. **The first clean zero-delay boot is a milestone worth naming.**

An environment that does not fail — it quietly stops checking

```
// `!==` is case inequality: its result is always 1'b0 or
// 1'b1, so an unknown in `act` always reaches the report.
// `!=` is logical inequality -- 1'bx when the relation is
// ambiguous -- and `if (1'bx)` takes the false branch,
// scoring the transaction as correct.
always_comb begin
    if ($isunknown(act))
        $error("unknown in sampled data: %0b", act);
    else if (act != exp)
        $error("mismatch: got %0h, expected %0h", act, exp);
end
```

A 2-state variable holding that result would convert the `x` to `0` by a second route.

The language rule is exact, and the exactness matters. For the **logical** equality and inequality operators, the result is `1'bx` only when unknown or high-impedance bits make the relation *ambiguous*. The **case** operators include those bits in the comparison and always yield a known result.

So `!=` does not simply "miss unknowns". A value corrupted only in bits that already differ from the expectation is still reported, because the relation is not ambiguous. What escapes is **the fully corrupted value**, and the partially corrupted one whose unknowns sit precisely on the bits that would have carried the difference.

WHY THE DEFECT SURVIVES REVIEW

The checker works on most of the data most of the time, and fails exactly where the data is worst. It fails **silently** — the run is green, the coverage is collected, the plan row is discharged. *Audit this before a testbench is ever pointed at a netlist.* The cure is two lines: an explicit unknown check ahead of the comparison, so that "unknown" is its own verdict rather than a special case of "equal".

SAY This one is worth a slow read. Two hours of a nine-hour debug went into a checker that was reporting success.

Three ways annotation lies quietly, and the artifact that catches each

RTL has no delays. A netlist has library delays — estimates for an average output load, while every instance of the same cell drives a different number of inputs over wires of different lengths. Back-annotation replaces them with per-instance values. What SystemVerilog takes from the file is **precisely bounded**: specify path delays, specparam values, timing-check constraint values, and interconnect delays. *Everything else in an SDF file is ignored.*

THE LIE	THE MECHANISM	THE DEFENCE
The un-annotated island	Any value the file does not supply retains its pre-annotation value, and the annotator need only <i>warn</i> . A mis-scoped read call leaves a whole subsystem on library defaults — frequently zero. Nothing fails . The run is fast, clean and meaningless for that block.	The annotation log is an <i>artifact to review</i> : each annotation produces an entry. Compare its instance count against the netlist's.
The corner nobody chose	Which member of each min/typ/max triple is annotated is a string argument whose default leaves the choice to the simulator. The one property deciding whether the run says anything about setup or about hold is a defaulted parameter of a system task , inherited from a script nobody has read in two years.	A run whose corner nobody selected is not evidence about any corner. Name it in the run manifest , beside the seed and the tool build.
The cost that shapes the flow	Millions of gates, millions of pin-to-pin connections, each needing annotation. Slow enough to be worth minimising.	Annotate <i>once</i> at compile time and run many tests against the same model — which is why the tier is a few long runs, and why it fights per-test randomisation.

SAY None of these three produces an error message. All three produce a green run that means nothing.

A timing check prints. The *notifier* is what makes a violation functional.

```
// Abridged library flip-flop model. Rows handling unknown inputs
// are elided; in a real cell they outnumber these, and they are
// where gate-level pessimism comes from.
primitive dff_udp (q, clk, d, notifier);
  output q; reg q; input clk, d, notifier;
  table
    // clk d ntf : state : q
    r 0 ? : ? : 0 ;
    r 1 ? : ? : 1 ;
    f ? ? : ? : - ;
    ? * ? : ? : - ;
    ? ? * : ? : x ; // notifier event => q goes x
  endtable
endprimitive
module dff_with_timing_check (output wire q, input wire clk,
                             input wire d);
  reg notifier;
  dff_udp u_ff (q, clk, d, notifier);
  specify
    specparam tSU = 120, tHLD = 40; // SDF overwrites per instance
    $setuphold(posedge clk, d, tSU, tHLD, notifier);
    (posedge clk => (q += d)) = 90;
  endspecify
endmodule
```

Every timing check may take a notifier — a variable the check toggles on violation, so the model can make its behaviour a *function* of violations. The canonical use: drive the output to `x`.

WHY GATE-LEVEL DEBUG FEELS THE WAY IT DOES

A timing violation is not a message you note and move past. It **injects an unknown into the datapath**, which propagates and produces a functional failure somewhere else entirely, thousands of cycles later. "Twelve timing violations, all in one block, early in the run" is never benign — it is the *cause* of whatever the scoreboard reports at the end. **Resolve timing-check output first, in time order.**

STA is the authority; GLS samples

But the constraints file STA reads is *manually generated*, and every exception in it — false path, multicycle path, case analysis — is a human claim STA **obeys rather than checks**. A back-annotated run makes no such assumption: **not a check on the timing, but a check that the constraints described the design you actually built.**

SAY The multicycle path declared correct at 8-bit weights is single-cycle at 2-bit weights. STA reports timing met, because it was told not to look. Run the configurations the constraints assumed something about.

Nobody wrote those checks. The corruption *is* the checking mechanism.

PD_AON — always on
power controller, wake logic, 32 kHz

isolation
clamp to a chosen value

PD_SYS — switched
corrupted to X while powered down; retention registers restored on exit

HDLs were built to capture what a design *computes*. They have no way to say how each element is *powered* — and the same RTL is instantiated into different power architectures by different integrators, so power intent must live outside the golden design description.

Five words, pinned precisely. A *power domain* is instances grouped for power management, sharing a supply set. *Isolation* gives defined behaviour for a signal whose driver is not active, via an *isolation cell* that clamps to a specified value. A *level-shifter* translates between voltage swings. *Retention* lets selected sequential elements survive power-down. A *power state table* states the legal combinations.

WHAT LOADING POWER INTENT CHANGES

Every logic node in a powered-down domain is **corrupted to unknown** and stays unknown until power returns. An isolation cell stops that unknown reaching powered-on logic. A retention register is corrupted like everything else, except that the value saved before entry is restored on exit.

So a missing isolation cell is not diagnosed by a rule — it is diagnosed by **the failure it now causes**: the unknown appears on the crossing, and downstream logic starts misbehaving shortly after the driving domain goes down.

Simstate makes the level of corruption a declared property: `NORMAL` →

`CORRUPT_STATE_ON_CHANGE` → `CORRUPT_STATE_ON_ACTIVITY` → `CORRUPT_ON_CHANGE`
→ `CORRUPT_ON_ACTIVITY` → `CORRUPT`. State `ON` carries `NORMAL`; `OFF` and `ERROR` both carry `CORRUPT`.

SAY Read the simstate ladder as a question you are being made to answer about every state you designed: does logic still switch here, does it hold without switching, or can it not even hold?

The ordering is not a convention. It is what the hardware requires.

POWER-DOWN	POWER-UP
1. Quiesce: no outstanding bus transactions	1. Assert the domain reset, <i>while power is still off</i>
2. Gate the domain clock	2. Close the header switch; wait for supply-good
3. Assert <code>save</code> : retention captures state	3. Release reset, <i>then</i> assert <code>restore</code>
4. Assert isolation: outputs clamp	4. Release isolation
5. Open the header switch: power off	5. Un-gate the clock

THREE OF THESE STEPS CARRY PUBLISHED FAILURES

Isolation must be asserted *before* sleep enable and released *after* it — swap either and the always-on island samples an unknown for the overlap. Reset must already be asserted when the supply returns: a design asserting reset *after* sleep-enable de-asserted showed the register output corrupted for exactly the window between supply arriving and reset landing. And reset must be inactive at the restore edge — which is why step 3 has an order *inside* it.

Where inspection stops working

2 states — reference SoC: one switchable domain. One power-down sequence, one power-up sequence, reviewable by three people around a whiteboard in an afternoon. *The table is documentation; directed tests check the sequences.*

128 combinations — flagship: six domains, DVFS giving the host cluster and the compute island four states each, the other three on or off. $4 \times 4 \times 2 \times 2 \times 2$. *An upper bound, not a count of legal states* — but nobody reviews 128 by inspection, and nobody enumerates the **transitions**, which is where the sequencing bugs live. The table becomes machine-readable and the checks are *generated* from it.

4,000+ legal states — a published 2010 case at **Samsung Electronics**: 40 million gates, 30 power domains, eight software-controlled, more than 2,000 lines of power intent, power-state coverage named as a challenge with no satisfactory tooling answer. *Coverage of the state space stops being achievable: prove the controller only emits legal sequences, and verify domains against those sequences.*

Scale and the coverage challenge above; the three sequencing failures opposite: *Low Power Verification with UPF*, Samsung Electronics with Synopsys, DVCon 2010 — [D17, pp. 4, 5, 8]. Self-reported, and scale is all it measures — no before/after payoff, so nothing here says the checklist made anything faster. The two sequence columns are the reference SoC's, not Samsung's.

SAY *Two, 128, four thousand: the same discipline, three regimes — and the technique has to change at each step.*

The assertion that would have caught it needs no stimulus at all

```
// Bound in the always-on domain: every signal here is always-on, so the
// checker runs while the switched domain has no power. 2 of 4 shown.
module pd_sys_sequence_chk (
    input wire clk_aon, rst_aon_n, // 32 kHz clock, async reset
    input wire iso_en,           // 1 = switched outputs clamped
    input wire sleep_en,         // 1 = the power switch is open
    input wire restore);         // retention restore, active high
    // Power may be removed only from an already-isolated domain.
    property p_iso_before_sleep;
        @(posedge clk_aon) disable iff (!rst_aon_n) $rose(sleep_en) |-> iso_en;
    endproperty
    a_iso_before_sleep : assert property (p_iso_before_sleep);
    // The clamp must hold for the whole restore window: the retention
    // strategy's restore-period condition, as an assertion.
    property p_restore_isolated;
        @(posedge clk_aon) disable iff (!rst_aon_n) restore |-> iso_en;
    endproperty
    a_restore_isolated : assert property (p_restore_isolated);
endmodule
```

The mechanism. The retention strategy carries a *restore-period condition* — "isolation still asserted" — and the standard is normative: it shall hold throughout the restore period, *otherwise the restore fails and the retention element is corrupted*. A fast-wake optimisation drops isolation the cycle a wake event arrives, once supply-good is asserted. The first wake cannot fire it: supply is still off. A **second** wake inside the restore window is qualified, so the bypass fires, the condition goes false mid-restore, and the interrupt mask is destroyed.

Plain RTL simulation cannot represent this; power-aware simulation can, but needs stimulus that hits the race. **An assertion needs neither** — it watches the controller's *outputs*, so it runs unchanged at RTL and is an excellent formal target.

POWER-AWARE SIMULATION HAS ITS OWN OPTIMISM

Results reflect the implemented hardware only so far as **the declared simstate is correct**. Some mismatches are conservative; others are optimistic, producing errors in the implemented design that the simulation showed working.

SAY Retention is a default to be argued against as well as for. Retain the crypto key registers and the engine works better — and the entire security value of powering the domain down is gone.

Cost and calendar are different numbers — and the difference is the whole decision

Month nine: **180 tests, 900 runs, 11-minute mean, 165 CPU-hours on 24 licences**, inside a ten-hour night. The same 180 at gate level, at **40×**: 7.3 h per run, **1,320 CPU-hours, 55 hours of wall clock**. At **10×**: 330 CPU-hours, **13.8 hours**, still outside the night. Break-even is around **7×** — dead across the plausible range.

THE SAME RATIO, FOURTEEN TESTS – READ THIS TWICE
At 40×, a gate-level tier of fourteen tests costs **102 CPU-hours**. But **fourteen runs into twenty-four licences is one wave**, so the wall clock is a single run's **7.3 hours** and ten slots stay free. *Cost is not calendar*: divide CPU-hours by licences and you get a number that describes nothing.

THREE INDEPENDENT CHOICES, NOT ONE DIAL	THE QUESTION IT ANSWERS	WHERE IT BELONGS
RTL + power intent	Does the power sequence work? Every crossing protected?	Nightly, from the day the intent exists
Zero-delay netlist	Does it leave reset with no unknowns? Test structures inert in functional mode?	Weekend, from first netlist
Zero-delay + power intent	Does the <i>implemented</i> protection logic match the intent?	Weekend, once cells are inserted
SDF-annotated, named corner	Do the timing constraints describe the design that was built?	Campaign, per corner
Post-layout + power intent	Are the always-on routes really always-on?	Campaign, per netlist drop

SAY The first row is the one teams get wrong most expensively: power-aware verification is an RTL activity that also runs at gate level. What never belongs, whatever the budget, is the constrained-random regression — randomness pays in volume, and volume is what the slowdown removes.

20

Post-Silicon Validation

Chapter 3's budget, exactly inverted. Pre-silicon you can see everything and run almost nothing; post-silicon you can run everything and see almost nothing.

Two silicon bring-ups, eleven months apart

Reference SoC — a part on a bench

One part, a JTAG probe, a scope on the 32 kHz crystal, a serial terminal. Thursday: powers on, releases reset, boots from ROM, blinks a GPIO. The following Friday: the DMA has moved a 2D tile and the UART echoes at 115,200 baud. **Two engineers, six weeks.** When something misbehaves they halt the core, read `STATUS` through the debug module — and, with 512 KB of SRAM and a handful of peripherals, usually guess right on the second try.

Flagship — the failure that costs three weeks

Under one mixture of decode and live network traffic, **roughly once every forty minutes, the system stops.** No exception, no error interrupt, no kernel log. The host cluster is idle, two NoC queues are full, the DMA reports no error on any channel, and every configuration register reads back exactly what firmware wrote.

THE ARITHMETIC THAT FRAMES THE WHOLE CHAPTER

Forty minutes at target speed is **of order 10^{12} cycles** — 2,400 seconds at any point on the flagship's **0.6–1.5 GHz** DVFS band, so every operating point lands on the same order. The on-chip trace buffer holds about **four thousand** of them.

What eventually closed it was not a debug technique. It was a decision taken **fourteen months earlier**, in a meeting nobody remembers, about which buses would get monitors, how deep the trace buffer would be, and whether those monitors would still be reachable when the compute island was power-gated.

SAY The uncomfortable centre of this chapter: the observability you have in the lab is the observability you designed in, months before you knew what would break.

Validation begins when the thing under test is a physical object

	PRE-SILICON (RTL SIMULATION)	POST-SILICON (A PART)
Cycles available	~nothing. A boot that takes seconds on silicon takes years on an RTL simulator	everything. Real workloads, real software stacks, indefinitely
Internal state visible	all of it , any signal, any cycle	a few hundred signals, for a few thousand cycles, <i>chosen in advance</i>
Stimulus reachable	anything the interface can express	whatever the running system and its configuration hooks can produce
What a fix costs	recompile	a new silicon stepping
What only this platform sees	nothing physical	power, temperature, electrical margin, real signal integrity, real peripherals

READ THE FIRST TWO ROWS TOGETHER

Their product is the whole subject. **Pre-silicon you can see everything and run almost nothing; post-silicon you can run everything and see almost nothing.** Every technique in this chapter buys back a little of the second row using the surplus in the first.

Three currencies, and the third reorganises the work. *Observability:* silicon exposes what DfD routes to a pin or a buffer, and nothing else. *Controllability:* exactly the number of configuration options the architecture defined — where a testbench has a **force**, the lab has a defeature bit, if someone put one in. *Fix latency:* a hardware fix needs a stepping, so debug runs **in parallel, on a shared part, with a growing stack of workarounds underneath it** — and each workaround must eliminate its bug's effect without masking the bugs still waiting to be found.

SAY The outside world arrives for the first time: 2017 compatibility work on a large SoC routinely spanned more than a dozen OS flavours, over a hundred peripherals and more than five hundred applications. That space was never simulable.

The goal of post-silicon debug is *not* to root-cause the bug

The goal is to narrow an observed failure down to an error scenario that can be investigated **in a pre-silicon environment**: a stimulus short enough to simulate, on a scope small enough to load, with the failure still present. Root-causing then happens where you can see.

Judge every technique in this chapter by how far it moves a failure toward that hand-off — not by how much it explains.

SCENARIO • THE LAB DOES NOT TRY TO EXPLAIN THE CORRUPTION. IT REDUCES.

The video codec passes every pre-silicon check, and a customer's live stream produces one corrupted macroblock roughly every two hours of playback.

Approach: capture the stream to memory; replay from DRAM instead of from the network; bisect to a 40-frame segment that still fails; then to a single frame; then hand the frame plus the decoder's register configuration to the pre-silicon team, who reproduce it in RTL simulation. **Why:** the oracle already exists and is golden — the bit-exact reference model runs on the captured stream on a workstation, so the lab can decide *corrupt or not* with no internal visibility at all. What the lab supplies is not a diagnosis but a **reproducer**, and each bisection step is worth more than any amount of staring at the trace buffer.

SAY And the calendar is fixed years in advance by market forecasts. Every technique here is judged first on how much calendar it saves.

One governing inequality: *signals × cycles ≤ buffer*

DfD is specified, sized, routed and taped out with everything else, **months before the first failure exists** — and adding it later requires a silicon spin. *There is no incremental path*: a prototype at least lets you re-select signals and rebuild at the cost of hours. The flagship's **64 KB** buffer is 524,288 bits, so the choice is not *how much* to observe but **width against depth on a fixed rectangle**:

128 signals × 4,096 cycles

256 signals × 2,048 cycles

512 signals × 1,024 cycles

INSTRUMENT	WHAT IT YIELDS	WHAT IT COSTS
Scan chains, reused for debug	full internal state at one <i>stopped moment</i>	already paid for by manufacturing test; needs a debug-usable stop
Signal trace into an on-chip buffer	a few hundred signals over a few thousand cycles, continuous	buffer area and power; routing congestion to the buffer
Processor trace	a core's instruction flow, compressed	modest per core; transport bandwidth
Bus and protocol monitors with triggers	transaction-level history at a chosen interface, on a condition	monitor logic plus a share of the same buffer
Hardware checkers in silicon	a stop <i>close to the bug's origin</i>	area, power, timing pressure — the reason there are few
Irritators, non-functional modes	<i>controllability</i> : forced corner conditions without the stimulus	area and power, plus a mode nobody may ever exercise

SAY Trace gives a short film of a few signals; scan a still photograph of nearly every flop — a lab that planned only one will find itself asking for the other. And why a buffer at all? The debug port runs at megahertz while the design runs far faster, so continuous export is arithmetically impossible.

Two conflicts DfD must survive — both settled a year before the lab exists

Power management

Debug data has to *travel*, and its route runs through logic power management may switch off. A block can be perfectly instrumented and still invisible because the path carrying its signals crosses an IP that is powered down at the moment of interest — **an IP with no functional relationship to the one under debug**. The reflexive fix, disabling power management during debug, destroys the ability to debug the power sequences themselves. The architectural fix is the flagship's: **one debug access port per power domain**.

SCENARIO • THE SENSOR HUB'S DAILY MISSED WAKE

The DfD route from the always-on island to the trace buffer passes through the DSP island — power-gated precisely while the bug is armed. What makes the debug possible: a debug access port *inside* the always-on domain, plus a small always-on capture of the four signals that matter, sampled at 32 kHz.

Observability of a domain must be powered by that domain. A trace path that depends on a neighbour's power state is observability you own only when you do not need it.

Security

DfD is, by construction, **a back door into the part's internal state** — and much of it stays present on-field so deployed systems can be diagnosed. That makes it an obvious path to the assets it sits beside: keys, firmware, debug modes. Published system compromises have used post-silicon observability features to get in. The answer is authenticated debug unlock through the root of trust, with the fabric's reach reduced once the part leaves the lab; the knee-jerk alternative, cutting DfD, buys security by making lab validation of the part impossible.

BOTH ARE RESOLVED BY PEOPLE WHO WILL NOT BE IN THE LAB

Post-silicon readiness — DfD architecture, debug software, debug boards, post-silicon test plans — is a *pre-silicon workstream* that starts at product-functionality planning and runs the whole length of the design cycle. Treat it like the verification plan: it has owners, rows and review gates, or it does not exist.

SAY On which signals to trace: **there is no known industrial report of the state-restoration ratio ever being used to select them**. Selection rests on experience — for each block, ask what failure you would most fear and what you would need to see to separate its two likeliest causes.

In the testbench you can add a probe. Here you cannot — so the *order* becomes the argument.

Each check must be decidable using only what the checks before it established. A step whose verdict depends on machinery not yet proven is a coin flip with two failure modes and no way to tell them apart.

#	CHECK	ORACLE AVAILABLE <i>HERE</i> — THE COLUMN THAT MAKES IT A BRING-UP PLAN
B05	One host core runs a three-instruction loop from boot ROM, writing a heartbeat register	the heartbeat toggles
B06	L2: write-read-verify a walking pattern across four slices	data comparison
B08	First timer interrupt taken and acknowledged on one core	a handler-incremented counter
B09	DMA channel 0: 1D copy, L2 to L2, one NoC hop	data comparison and completion interrupt
B10	OS boots to a shell on one core, then on four	console output

WHY THIS ORDER

B05 is the first row whose oracle lives *inside* the part, and deliberately the smallest available: a heartbeat that stops says the core stopped; a failing OS boot says almost nothing. **B08 follows B06** because a handler needs a stack and a stack needs memory. **B09 depends on both at once**, which is why it is late. **B10 is one row, not ten**: when the shell fails after every row above it passes, the search space is software.

SAY Every row also carries an exit criterion — twenty consecutive reads, twenty consecutive boots — because a step closed by “it seems to be running” is closed by fatigue. Earlier rows check the rails on bench instruments alone, and before all of it comes power-on debug, with no DfD at all.

Detection is often free. Localisation is what costs the calendar.

The governing quantity is **error detection latency**: the time between a test activating a bug and that error becoming an observable failure. For difficult bugs it runs to *millions or even billions* of cycles.

NOW PUT THAT NEXT TO THE RECTANGLE

The run is of order **10¹²** cycles; the buffer at 128 signals wide holds **4,096**. The trace covers roughly **one part in 250 million** of the execution — a figure deliberately conservative against the clock band. And placement is not free: even a perfect trigger capturing the last 4,096 cycles before the hang captures the *symptom*, while the activation sat millions of cycles earlier, outside the window *by construction*.

A checker tells you the answer is wrong. The trace buffer shows you four thousand cycles of a forty-minute film, and the interesting scene is not in them.

STAGE	WHAT HAPPENS → EXIT
Test execution	run it; on failure, the obvious sanity checks → resolved, or a <i>pre-sighting</i>
Pre-sighting analysis	make it repeatable under varied conditions → a <i>sighting</i>
Sighting disposition	assign a team; agree a plan → an owned plan
Bug resolution	work around, group, localise, root-cause, prove the fix → a <i>bug</i> , resolved

The first classification question is one no pre-silicon triage rota ever asks: **is this a silicon issue at all, or a logic error?** A failure that follows the part is *manufacturing or marginality*; one that follows the design is yours.

THE WEAKENED STANDARD, AND WHY IT IS A GOOD TRADE

Because silicon is fast, an error appearing reliably **once in a few executions still counts as reproducible**. A one-in-five rate is a flake in a nightly regression; in the lab it is a working recipe, because five runs cost minutes.

SAY "Cannot reproduce" almost never means non-deterministic — it usually means the recipe has a parameter nobody has varied. The radar's spurious detections appeared only above ~85 °C and only on parts calibrated in the lower third of the distribution: two conditions no bench at room temperature produces together.

Four ways to attack a window that is 250 million times too small

1 • Bisect the configuration

one run per question

→

2 • Bisect in time

many forward runs, stitched

→

3 • Shorten the latency

software-only transformation

→

4 • Move to a platform that can see

the hand-off

1. Configuration bisection is cheap, coarse and criminally under-used. If the failure needs four cores, does it need three? One run per question — and it decides where the expensive instruments point. The GPU: clusters {0,1} fail; {0,2}, {1,3}, {2,3} do not; {0,1} with the DMA idle does not. **Five runs** reduce "somewhere in the GPU" to "two clusters sharing a port into unified memory, plus DMA traffic".

2. COST THE TIME BISECTION HONESTLY

Narrowing an order- 10^{12} run to one 4,096-cycle window takes $\log_2(10^{12}/4,096) \approx 28$ **halvings** — at forty minutes a run, **about nineteen hours** of pure execution, and over two days if it reproduces one run in three. And the search is *two-dimensional*. **A buffer twice as deep removes one halving out of twenty-eight.** What collapses the cost is a *trigger*.

3. Attack the latency, not the search. Quick error detection shortens the interval between activation and observable failure — a read after each write catches a wrong value at once — and the transformations are *software-only*. Bounded model checking on top turns detection into localisation: on the **OpenSPARC T2**, a 500-million-transistor open-source multicore SoC, difficult logic bugs were localised automatically in **under seven hours**, with activating counter-examples **up to six orders of magnitude** shorter.

4. THE FORMAL HAND-OFF, AND WHY IT IS UNDERRATED

Do not take the failing sub-system as the proof scope. Pick the *smallest block boundary at which you can still observe the property*; over-constrain everything that did not participate; leave the signals of interest under-constrained; load state from a waveform after initialisation. Reported at **Intel**: a post-silicon bridge failure reproduced and root-caused in a formal environment built from scratch in **three person-days on a 25K-gate DUT**, the fix validated in **five person-days** more, and the same environment reused on two later SoCs for **eight further functional bugs**.

SAY *The plan row a silicon escape indicts is, by revealed preference, the row that deserves formal sign-off in the next generation.*

3: Lin, Singh, Barrett, Mitra, Stanford/NYU, ITC 2015 — [P9, pp. 1–2]; no originating company is named for the T2. 4: Jain et al., Intel, DVCon — [D3, p. 7]; no product named.

The lab's bug list is an unfiltered report card, written by reality, arriving too late to argue with

LINK THAT BROKE	WHAT IT MEANS	THE UPSTREAM CHANGE IT OWES
Activation	the stimulus never created the condition	a new plan row, and a cross that would have demanded it
Propagation	it happened but reached no observable — the platform did not model the mechanism	a platform change: a level, an engine, or an earlier scheduling
Checking	it happened, it was visible, nothing looked	a new checker, plus a test that proves the checker fires
Assumption	an <code>assume</code> that does not hold deletes traces <i>silently</i> ; nothing reports it	not a checker or a bin — a written assumption promoted to a checked obligation

Which question bites changes. Pre-silicon escapes are usually stimulus escapes. Post-silicon escapes concentrate in "why did the metrics say we were done?" — the dashboards were green. **An escape analysis that cannot explain a green dashboard has not finished.**

THE CODEC ESCAPE RECORD

Why no stimulus? The corpus has streams with unusual slice geometry and streams with frequent reference-list updates — *none with both*: it is organised by feature.

Why were the metrics green? A coverpoint for slice geometry, one for reference-list update, both at 100%. **No cross.**

What changes: the cross; two synthesised streams; a plan-row family for stream-syntax *combinations*; and a stream generator wherever the oracle is free. *The first two fix the bug; the last two pay.*

SAY *The best-documented public programme sets the bar high enough to sting: on IBM POWER8, a bug first found by an OS-based test or by real software counted as an escape from the validation strategy — not a success. "We already found that one" is a claim the next generation must earn.*

Mishra, Ray, Morad, Ziv, *Post-Silicon Validation in the SoC Era*, IEEE Design & Test, 2017 — [P21, p. 20]. Self-reported: two authors are IBM.

Pre-silicon: find → root-cause → fix → verify. Post-silicon, the workaround comes first.



OPTION	WHEN IT WINS	WHAT IT OBLIGES
New stepping	no workaround exists or its cost is unacceptable; safety- or security-relevant; blocks a committed use case	the fix proven pre-silicon <i>before</i> the masks change
Firmware workaround	a configuration or code path can avoid the condition reliably	proof it covers every path to it, and a named owner
Errata	outside committed use cases, or the customer can avoid it	a named workaround, an owner, a decision with a real expiry

THE TWIN OF THE DFD SECTION

The workaround must eliminate the bug's effect **without masking the bugs still waiting to be found**. And patchability depends on configurability built in beforehand: **DfD buys sight; design for configurability buys a way out**.

WHAT VERIFICATION OWES THE ROOM

What the bug does. What conditions are necessary and sufficient to hit it.
What the workaround does and does not cover. A team that supplies a severity label and a shrug has outsourced the one judgement it was qualified to make.

SAY *Severity is not a property of the symptom: a hang may be one register bit from a free firmware workaround; a one-bit corruption may have none. The AES-GCM key-slot bug went to a stepping because a workaround depending on every future software author is not a workaround for a security property.*

The through-line: every platform past RTL simulation trades sight for reach

Chapter 17 buys cycles and charges visibility, turnaround and debug ergonomics. Chapter 18 buys reality and charges nearly all the visibility left. Chapter 19 buys the netlist's truth and charges runtime. Chapter 20 buys everything and charges almost all of it.

- 17 Acceleration and emulation — the boundary and what earns a slot
- 18 FPGA prototyping and co-verification — and who owns the bugs
- 19 Gate-level, timing, power-aware — exhibit, do not sign off
- 20 Post-silicon validation — the observability you designed in

WHAT PART V DELIBERATELY LEFT OPEN

The analog and mixed-signal boundary neither a prototype nor an RTL model can host (Ch 21). The debug-unlock path as an attack surface (Ch 23). Manufacturing test itself. And the capital decision: what a machine is *for*, not whether to buy it.

SEVEN SENTENCES WORTH LEAVING WITH

- Is my problem the number of runs, or the length of one run?
- Anything the environment needs every cycle becomes hardware.
- A prototype is pre-silicon in time and validation in kind — what you see is decided before the run.
- Exhibiting a failure and signing off its absence are different jobs.
- A run whose corner nobody selected is not evidence about any corner.
- Cost is not calendar: fourteen runs into twenty-four licences is *one wave*.
- The observability you have in the lab is the one you designed in, before you knew what would break.